4장

조건에 따라 해야 할 일이 다를 때: 조건문

프로그램의 기본 요소인 명령어와 데이터를 저장하는 변수를 배웠습니다. 또 하나 배운 것이 있죠. 바로 제어구조입니다. 제어구조는 어떤 특정한 순서 혹은 어떤 조건에 따라 실행되는 명령어들의 흐름을 제어하는 구조입니다. 순차구조는 위에서 아래로 순서대로 실행되는 구조입니다. 순차구조는 건너뛰거나 같은 명령을 반복하지 않는 가장 간단한 제어구조입니다.

4.1

조건이 하나일 때: if

4.1.1 선택구조란

특정 조건에서만 명령어가 실행되는 구조를 선택구조라고 합니다. 일상생활에서 우리는 다양한 선택구조를 접합니다. 예를 들어, 주류는 미성년자에게 팔 수 없고 성인에게만 판매할 수 있습니다. 명령형으로 표현하면 '성인일 때만 주류를 판매하라'입니다. 이때 조건은 '성인'이고, 실행할 내용은 '주류 판매'입니다.

```
성인일 때만 주류를 판매하라
```

간단하게 '조건 A를 충족할 때만 B를 하라'고 정리할 수 있습니다.

```
조건 A를 충족할 때만 B를 하라
```

다른 예를 볼까요? 온라인 쇼핑몰에서 총 구매금액이 5만 원 이상일 때만 무료배송이 되는 경우를 보았을 거예요. 이를 선택구조로 만든다면 '총 구매금액이 5만 원 이상일 때만 무료배송을 하라'가 됩니다.

```
총 구매금액이 5만 원 이상일 때만 무료배송을 하라
```

아침을 시작하는 알람에도 선택구조를 적용할 수 있습니다. 주중에는 학교나 직장에 가기 위해 일찍 일어나야 하고, 주말에는 늦잠을 자면서 여유를 부립니다. 주중을 기준으로 선택구조를 만들면 '주중일 때만 아침 7시에 알람을 울려라'가 됩니다.

```
주중일 때만 아침 7시에 알람을 울려라
```

선택구조가 어떤 것인지 알겠죠? 그럼 파이썬에서는 선택구조를 어떻게 표현하는지 배워 봅시다.

4.1.2 선택구조를 조건문으로 표현하기

조건문의 형식

'조건 A를 충족할 때만 B를 하라'는 선택구조를 파이썬에서는 다음 형식으로 표현합니다. `if`와 콜론(:) 사이에 조건을 넣고, 다음 줄에 실행할 내용을 넣습니다. 이를 조건문이라고 합니다.

형식

```
if A: # 조건
    B # 실행할 내용
```

TIP 2장에서 컴퓨터는 명령어만 알아듣기 때문에 미리 정의된 명령어를 사용해야 한다고 했습니다. 명령어는 대부분 앞에서 배운 `print()`와 `input()`처럼 영단어 뒤에 소괄호가 붙는 형태지만, `if`처럼 프로그램의 흐름을 조절하는 명령어는 영단어 뒤에 괄호가 붙지 않기도 합니다. 형식에 맞게 입력하면 에디터에서 해당 명령어 부분이 강조 표시되어 미리 정의된 명령어임을 알 수 있습니다.

그럼 앞서 예로 든 사례를 파이썬 코드로 작성해 조건문을 어떻게 만드는지 알아봅시다.

먼저 편의점에서 주류를 판매할 때 성인이면 주류를 구매할 수 있다고 출력해 봅시다. 코딩하기 전에 어떻게 작성할지 구상해 보죠. 일단 성인인지 구분하려면 나이를 알아야 합니다. 따라서 나이를 저장하는 변수가 있어야죠. 변수명은 `age`로 정합시다. 조건문 형식에서 조건(`A`)과 실행할 내용(`B`)에 해당하는 부분을 찾아봅시다. 조건은 '성인'이고, 조건을 충족할 때 실행할 내용은 '주류를 구매할 수 있다고 출력하기'가 됩니다. 정리하면 다음과 같습니다.

조건문 형식	형식에 맞춰 조건과 실행할 내용 넣기	실제 코드
`if A:` `  B`	`if 성인:` `  주류를 구매할 수 있다고 출력하라`	`if age > 19:` `  print('성인은 주류를 구매할 수 있습니다.')`

조건과, 조건을 충족할 때 실행할 내용만 정하면 실제 코드로 옮기는 것은 어렵지 않습니다. 변수 `age`를 만들어 23을 넣은 후 조건문을 작성해 봅시다.

```
age = 23
if age > 19:
    print('성인은 주류를 구매할 수 있습니다.')
```

```
실행결과
성인은 주류를 구매할 수 있습니다.
```

`age`에 저장된 값이 23이므로 조건인 `age > 19`를 충족해 `print()` 명령어가 실행됩니다. 오타가 없고, 조건문의 형식을 잘 지켰다면 에디터에서 `if`가 강조 표시됩니다. 처음 `if` 조건문을 실행하면 다양한 오류가 발생할 수 있습니다. 가장 흔한 오류는 조건문에서 콜론(`:`)을 빠뜨린 경우입니다.

```
age = 23
if age > 19
    print('성인은 주류를 구매할 수 있습니다.')
```

```
실행결과
if age > 19
           ^
SyntaxError: invalid syntax
```

`SyntaxError`가 발생합니다. `SyntaxError`는 유효하지 않은 문법(invalid syntax) 즉, 문법 오류를 의미합니다. 오류 내용을 보면 조건문을 작성하는 데 필수 요소인 콜론이 빠졌음을 알려 주고 위치까지 표시합니다.

콜론이 있는데도 오류가 나는 경우가 있습니다. 이때는 `if` 아랫줄에 실행할 내용을 작성할 때 들여쓰기를 제대로 했는지 살펴보세요.

```
age = 23
if age > 19:
print('성인은 주류를 구매 수 있습니다.')
```

```
실행결과
print('성인은 주류를 구매할 수 있습니다.')
    ^
IndentationError: expected an indented block
```

`IndentationError`는 들여쓰기(indentation)를 잘못했을 때 발생하는 오류입니다.

일반적으로 에디터에서 `if` 문을 작성하고 Enter를 누르면 다음 줄에서 자동으로 들여쓰기된 후 커서가 깜빡입니다.

```
if A:
    B
4칸 들여쓰기
```

들여쓰기

조건문 형식을 보면 조건 아래에 실행할 내용을 넣을 때 4칸 들여쓰기를 했습니다. `if A: B`처럼 한 줄로 작성해도 조건문은 오류 없이 실행됩니다. 그러나 명령어를 줄바꿈해 작성하는 것이 일반적입니다.

파이썬의 조건문에서는 이렇게 한 덩어리의 명령어를 줄바꿈하는 경우에 반드시 들여쓰기를 해야 합니다. 들여쓰기를 해서 명령어가 조건문 안에 속한다는 것을 시각적으로 표시하죠. 앞의 코드에서 `if` 문 아래에 `print()`를 들여쓰기 하면 `print()` 명령어가 `if` 조건문과 한 덩어리(블록)라고 인식합니다.

프로그래밍 언어마다 들여쓰기 기준은 다르지만, 파이썬에서는 4칸 들여쓰기가 일반적입니다. SpaceBar를 네 번 또는 Tab을 한 번 누르면 됩니다. 컴퓨터 설정에 따라 Tab이 4칸이 아닌 경우도 있습니다. 구글 코랩에서는 설정에서 들여쓰기 너비를 조정할 수 있습니다(1장 '코랩 사용하기'를 참고). 대부분 에디터는 콜론 다음에 Enter를 누르면 자동으로 들여쓰기가 됩니다.

이번에는 반대로 미성년자일 경우 주류를 구매할 수 없다고 출력해 보겠습니다. 앞의 조건문에서 조건(`A`)과 실행할 내용(`B`)을 수정하면 되겠죠? 조건이 미성년자이므로 조건을 `age < 20`로 바꾸고, 조건을 충족할 때 실행할 내용도 다음처럼 바꿉니다.

```
age = 18
if age < 20:
    print('미성년자는 주류를 구매할 수 없습니다.')
```

```
실행결과
미성년자는 주류를 구매할 수 없습니다.
```

변수 `age`에 18을 넣고 실행하면 조건을 충족하기 때문에 경고 문구가 출력되는 것을 볼 수 있습니다.

앞의 코드는 변수 `age`가 `18`로 고정되어 있습니다. 이번에는 나이를 직접 입력받아 `age`에 저장하게 바꿔 봅시다. 바뀐 코드를 바로 보지 말고 먼저 스스로 코드를 수정한 후 본인의 생각이 맞았는지 확인해 보세요.

```
age = int(input('나이가 어떻게 되세요? '))
if age < 20:
    print('미성년자는 주류를 구매할 수 없습니다.')
```

`input()`으로 값을 입력받아 변수 `age`에 저장합니다. 이때 어떤 값을 입력받는지 알기 쉽도록 `input()`에 안내문을 넣습니다. `input()`은 입력받은 데이터를 모두 문자열로 인식합니다. 따라서 숫자와 비교하려면 `int()`로 감싸 데이터 타입을 정수형으로 바꿔야 합니다.

입력칸에 20 미만인 수와 20 이상인 수를 하나씩 넣어 각각 실행해 봅시다.

```
실행결과
19를 입력했을 때
나이가 어떻게 되세요? 19
미성년자는 주류를 구매할 수 없습니다.

22를 입력했을 때
나이가 어떻게 되세요? 22
```

`19`를 입력하면 조건을 충족하므로 `print()` 명령어가 실행되어 '미성년자는 주류를 구매할 수 없습니다.'가 출력됩니다. 그러나 `22`를 입력하면 조건을 충족하지 않으므로 아무일도 일어나지 않고 프로그램이 종료됩니다. 이처럼 입력받은 값에 따라 실행결과가 달라집니다.

```
20 미만인 수를 입력했을 때
age = int(input('나이가 어떻게 되세요? '))                  첫 번째 실행
if age < 20: → 조건을 충족하므로 조건문 내부 명령어 실행   두 번째 실행
    print('미성년자는 주류를 구매할 수 없습니다.')          세 번째 실행

20 이상인 수를 입력했을 때
age = int(input('나이가 어떻게 되세요? '))    첫 번째 실행
if age < 20: → 조건을 충족하지 않으므로 조건문 내부 명령어를 실행하지 않음   두 번째 실행
    print('미성년자는 주류를 구매할 수 없습니다.')
```

이번에는 조건문 내부에 명령어를 추가해 봅시다. 미성년자가 주류를 구매하러 왔을 경우 'n년 후에 성인이 되면 오세요!'라고 출력하게 하겠습니다(이때 n은 성인이 되는 해까지 남은 연수입니다). 먼저 스스로 코드를 작성해 본 후에 책에 나온 코드를 보세요.

```
age = int(input('나이가 어떻게 되세요? '))
if age < 20:
    print('미성년자는 주류를 구매할 수 없습니다.')
    print(20 - age, '년 후에 성인이 되면 오세요!')
```

입력에 따라 출력하는 `n` 값이 달라져야 합니다. 그러면 `n`은 어떻게 구할까요? 성인이 되는 나이인 20살에서 현재 나이를 빼면 몇 년 후에 성인이 되는지 알 수 있습니다. 따라서 `20 - age`를 하면 됩니다. 추가된 출력 명령어는 조건이 충족돼야만 실행되므로 `if` 문 아래에 첫 번째 `print()` 명령어 다음 줄에 4칸 들여쓰기를 해서 작성하면 됩니다.

```
실행결과
18을 입력했을 때
나이가 어떻게 되세요? 18
미성년자는 주류를 구매할 수 없습니다.
2 년 후에 성인이 되면 오세요!

22를 입력했을 때
나이가 어떻게 되세요? 22
```

들여쓰기를 하지 않고 작성하면 어떻게 될까요? 20 미만인 수와 20 이상인 수를 넣어 보며 한번 테스트해 보세요.

```
age = int(input('나이가 어떻게 되세요? '))
if age < 20:
    print('미성년자는 주류를 구매할 수 없습니다.')
print(20 - age, '년 후에 성인이 되면 오세요!')
```

```
실행결과
18을 입력했을 때
나이가 어떻게 되세요? 18
미성년자는 주류를 구매할 수 없습니다.
2 년 후에 성인이 되면 오세요!

22를 입력했을 때
나이가 어떻게 되세요?
-2 년 후에 성인이 되면 오세요!
```

18을 입력하면 제대로 결과가 나오지만, 22를 입력하면 음수가 출력됩니다. 프로그램을 재실행해 30을 입력해도 마찬가지입니다. `if` 문 아래에서 들여쓰기가 되면 조건문에 속하지만, 들여쓰기를 하지 않으면 조건문 소속이 아니어서 조건 충족 여부와 상관없이 실행됩니다. 그래서 20 미만인 숫자를 입력해도, 20 이상인 숫자를 입력해도 마지막 줄은 항상 실행됩니다.

그럼 나이와 상관없이 항상 '감사합니다. 안녕히 가세요.'를 출력해 볼까요?

```
age = int(input('나이가 어떻게 되세요? '))
if age < 20:
    print('미성년자는 주류를 구매할 수 없습니다.')
    print(20 - age, '년 후에 성인이 되면 오세요!')
print('감사합니다. 안녕히 가세요.')
```

마지막 줄에 들여쓰기를 하지 않고 출력 명령어를 넣어 주면 됩니다. 20 미만인 수와 20 이상인 수를 각각 입력해 보고 원하는 대로 결과가 나오는지 확인해 보세요!

```
실행결과
16을 입력했을 때
나이가 어떻게 되세요? 16
미성년자는 주류를 구매할 수 없습니다.
4년 후에 성인이 되면 오세요!
감사합니다. 안녕히 가세요.

40을 입력했을 때
나이가 어떻게 되세요? 40
감사합니다. 안녕히 가세요.
```

조건문과 들여쓰기 개념을 한 번 더 확인하기 위해 실행 순서를 살펴보겠습니다.

```
20 미만인 수를 입력했을 때
age = int(input('나이가 어떻게 되세요? '))                    첫 번째 실행
 if age < 20: → 조건을 충족하므로 조건문 내부 명령어 실행   두 번째 실행
    print('미성년자는 주류를 구매할 수 없습니다.')           세 번째 실행
    print(20 - age, '년 후에 성인이 되면 오세요!')           네 번째 실행
print('감사합니다. 안녕히 가세요.')                         다섯 번째 실행

20 이상인 수를 입력했을 때
age = int(input('나이가 어떻게 되세요? '))   첫 번째 실행
if age < 20: → 조건을 충족하지 않으므로 조건문 내부 명령어를 실행하지 않음 두 번째 실행
    print('미성년자는 주류를 구매할 수 없습니다.')
    print(20 - age, '년 후에 성인이 되면 오세요!')
print('감사합니다. 안녕히 가세요.')          세 번째 실행
```

들여쓰기를 하지 않은 코드는 항상 실행되고, 들여쓰기를 한 코드는 조건문의 조건이 충족될 때만 실행된다는 점 기억해 두세요.

4.1.3 조건문을 만드는 비교 연산자

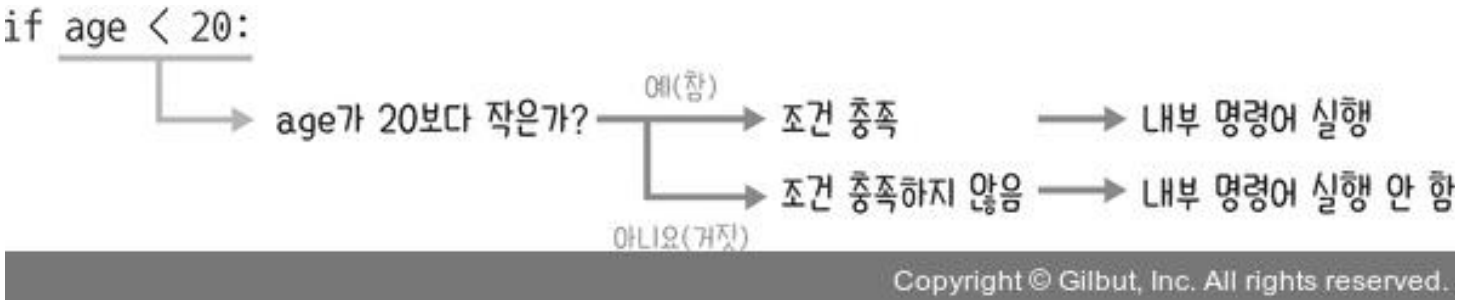
조건문의 형식을 배웠습니다. 조건 A가 충족될 때 B를 실행합니다.

형식

```
if A:
    B
```

A에는 다양한 조건이 들어갈 수 있는데, 이번에는 조건을 어떻게 작성하는지 알아보겠습니다.

앞의 예제에서는 `age < 20`처럼 조건에 부등호를 썼습니다. 수학에서는 이 식이 'age가 20보다 작다'는 의미입니다. 그런데 프로그래밍에서는 부등호가 조건에 쓰이면 의문문으로 해석하는 것이 더 정확합니다. 조건문은 조건 충족 여부에 따라 실행할 명령이 달라지므로 'age가 20보다 작은가?'라는 물음에 '예(또는 참)' 또는 '아니요(또는 거짓)'라고 답하는 것으로 조건의 충족 여부를 판단할 수 있습니다.



Copyright © Gilbut, Inc. All rights reserved.

이때 대답이 '예'면 조건문 내부의 `print()` 명령어를 실행하고, '아니요'면 실행하지 않습니다. 반대로 'age가 20보다 큰가?'라고 물어보고 싶으면 부등호 방향만 바꿔서 `age > 20`로 표현하면 되겠죠? 그럼 'age가 20보다 크거나 같은가?' 또는 'age가 20보다 작거나 같은가?'는 어떻게 표현하면 될까요? 이 역시 수학에서 배운 부등호와 비슷하게 각각 `age >= 20`, `age <= 20`로 표현합니다.

TIP 수학에서는 이상, 이하를 표현하는 부등호 ≥, ≤를 사용하지만, 키보드에는 이런 기호가 없습니다. 그래서 >, <와 =를 위아래가 아닌 옆으로 나란히 붙여 사용합니다. ≥는 >=로, ≤는 <=로 표현하죠. 이때 반드시 >, <가 =보다 앞에 나와야 합니다. =>나 =<는 잘못된 표기입니다.

‘age와 20이 같은가?’라고 물어보고 싶을 때는 어떻게 해야 할까요? `age = 20`으로 하면 될까요? 확인해 봅시다.

```
age = 10
if age = 20:
    print(age)
```

`실행결과`

```
if age = 20:
       ^
SyntaxError: invalid syntax
```

앞에서 봤던 `SyntaxError`가 발생합니다. 문법이 틀리면 발생하는 오류라고 했죠. 사실 `age = 20`은 앞에서 배웠습니다. 변수 `age`에 `20`을 넣을 때 `age = 20`라고 작성했습니다. 즉, `=`는 변수에 값을 저장할 때 사용하는 연산자입니다. 프로그래밍에서 ‘age와 20이 같은가?’라고 값을 비교할 때는 `=`를 2개 붙인 `==` 연산자를 써야 합니다. 코드를 수정한 후 다시 실행해 봅시다.

```
age = 10
if age == 20:
    print(age)
```

`age`에 들어 있는 값이 10이므로 ‘age와 20이 같은가?’에 대한 답이 ‘아니요’이므로 `print(age)`가 실행되지 않습니다.

그럼 'age와 20이 다른가?'는 어떻게 작성할까요? 이때는 `!=` 연산자를 사용합니다. `!`는 부정을 의미하므로 `=`에 `!`가 붙어서 '같지 않다'는 의미가 됩니다. 그럼 `!=`도 확인해 보겠습니다.

```
age = 10
if age != 20:
    print(age)
```

```
실행결과
10
```

'age와 20이 다른가?'라는 질문에서 대답이 '예'가 되므로 들여쓰기된 명령어 `print(age)`가 실행되어 10이 출력됩니다.

프로그래밍에서는 지금까지 배운 기호들을, 두 값의 크기를 비교하므로 비교 연산자라고 합니다. 비교 연산자는 다음 표와 같습니다. 표에 나온 비교 연산자를 알고 있으면 조건문에 필요한 조건을 쉽게 만들 수 있습니다.

▼ 표 4-1 비교 연산자

비교 연산자	의미	비교 연산자	의미
`a > b`	a가 b보다 큰가?	`a <= b`	a가 b보다 작거나 같은가?
`a < b`	a가 b보다 작은가?	`a == b`	a와 b가 같은가?
`a >= b`	a가 b보다 크거나 같은가?	`a != b`	a와 b가 다른가(같지 않은가)?

4.1.4 조건문으로 간단한 프로그램 만들기

무료배송

선택구조를 설명할 때 쇼핑몰의 무료배송을 예로 들었습니다. 이 부분을 프로그램으로 만들어 보겠습니다. 먼저 가격을 입력받고 입력 가격이 5만 원 미만이면 '배송비 2500원이 추가됩니다.'를 출력하는 코드를 작성해 봅시다.

> 제제쌤의 조언
> 조건문 형식에서 조건과 조건을 충족할 때 실행될 명령어가 무엇인지 잘 생각해 보세요. 앞서 작성한 편의점의 주류 판매 코드와 매우 유사하니 먼저 직접 작성해 본 후 책의 코드를 확인해 주세요!

```
price = int(input('구매금액은? '))
if price < 50000:
    print('배송비 2500원이 추가됩니다.')
```

```
실행결과
구매금액은? 35000
배송비 2500원이 추가됩니다.
```

구매금액을 입력받아 정수형으로 변환하고 `price` 변수에 저장합니다. 조건은 '5만 원 미만'이고, 조건을 충족할 때 실행되는 명령어는 '배송비 2500원이 추가됩니다.'를 출력하는 것입니다. 지금까지 설명한 `if` 조건문을 잘 이해했다면 어렵지 않을 겁니다. 다양한 숫자를 넣으며 결과를 확인해 보세요.

여기에 하나 더 추가하겠습니다. 구매금액이 5만 원 미만이라면 실제 결제금액은 구매금액에 배송비를 더한 금액입니다. 구매금액이 5만 원 미만일 때 구매금액에 배송비 2500원을 더해서 출력합니다. 마찬가지로 먼저 코드를 생각해 본 후 책을 보세요.

5만 원 미만일 때만 배송비를 더해야 하므로 조건은 앞의 코드와 같습니다. 따라서 추가하는 코드는 `if` 문에 속해야 합니다. 실제 결제금액은 구매금액을 저장한 `price` 변수에 2500원을 더해 다시 `price` 변수에 저장합니다. 따라서 `price = price + 2500` 또는 `price += 2500`으로 작성합니다. 실행해 봅시다.

```
price = int(input('구매금액은? '))
if price < 50000:
    print('배송비 2500원이 추가됩니다.')
    price += 2500
print('결제금액은', price, '원입니다.')
```

```
실행결과
35000을 입력했을 때
구매금액은? 35000
배송비 2500원이 추가됩니다.
결제금액은 37500 원입니다.

70000을 입력했을 때
구매금액은? 70000
결제금액은 70000 원입니다.
```

결제금액은 구매금액에 상관없이 출력돼야 하므로 조건문에 속하지 않게 마지막 줄에 들여쓰기를 하지 않고 넣습니다.

알람 설정

알람 예시도 코드로 작성해 봅시다. 사용자로부터 주중과 주말을 입력받아 값이 주중이면 '아침 7시입니다! 주인님 일어나세요!'를, 주말이면 '아침 10시입니다! 주인님 일어나세요!'를 출력합니다.

조건이 2개네요. 그럼 조건문도 2개가 필요합니다. 스스로 작성한 뒤에 책과 비교해 보세요.

```
day = input('오늘은 주중입니까? 주말입니까? ')
if day == '주중':
    print('아침 7시입니다! 주인님 일어나세요!')
if day == '주말':
    print('아침 10시입니다! 주인님 일어나세요!')
```

값을 입력받아 변수 `day`에 저장합니다(의미가 잘 드러난다면 변수명은 달라도 상관없습니다). 비교하는 값이 문자이므로 앞선 예제와는 달리 입력받은 값을 `int()`로 감싸지 않아도 됩니다. 그다음 조건문을 작성합니다. 입력받은 값인 `day`가 '주중'인지 '주말'인지에 따라 조건이 달라지므로 조건문은 2개를 작성합니다. 첫 번째 조건은 '변수 `day`의 값이 주중과 같은가?'입니다. 프로그래밍에서 같음을 나타내는 연산자는 `=`가 아니라 `==`라는 점을 주의하세요. 그리고 `day` 값이 주중이라면 아침 7시 기상이므로 조건문 내부에 `print()`를 넣습니다. 두 번째 조건도 비슷합니다. 조건을 주중에서 주말로 바꾸고, 출력할 문장에서도 7시를 10시로 바꿉니다.

실행해서 주중과 주말을 각각 입력하고 결과가 잘 나오는지 확인하세요.

```
실행결과
주중을 입력했을 때
오늘은 주중입니까? 주말입니까? 주중
아침 7시입니다! 주인님 일어나세요!

주말을 입력했을 때
오늘은 주중입니까? 주말입니까? 주말
아침 10시입니다! 주인님 일어나세요!
```

주중을 입력하면 첫 번째 `if` 문에서 조건과 같으므로 아침 7시 알람이 출력됩니다. 두 번째 `if` 문에서는 조건과 같지 않으므로 `print()` 명령어는 실행되지 않고 프로그램이 종료됩니다. 주말을 입력하면 첫 번째 `if` 문의 조건과 같지 않으므로 `print()` 명령어가 실행되지 않고 두 번째 `if` 문으로 넘어갑니다. 두 번째 `if` 문에서는 조건과 같으므로 아침 10시 알람이 출력됩니다.

1분 퀴즈

1 다음 중 옳지 <u>않은</u> 것을 고르세요.

① 어떤 조건을 충족할 때만 명령어가 선택적으로 실행돼야 할 때 조건문을 사용한다.

② 조건문의 형식은 `if A: B`이다.

③ 조건문을 사용할 때 콜론(`:`)을 넣지 않거나 들여쓰기를 잘못하면 오류가 발생한다.

④ '변수 a가 10과 같다'를 코드로 옮기면 `a = 10`이다.

⑤ '변수 a가 10과 같지 않다'를 코드로 옮기면 `a != 10`이다.

정답 및 해설

4.2

조건을 충족하지 않을 때: else

4.2.1 그렇지 않으면 else

앞에서 작성한 알람 설정 프로그램에서 주중과 주말, 두 가지 조건을 동시에 충족하는 경우가 있을까요? 당연히 없습니다. 일주일은 주중과 주말로 나뉩니다. 그럼 '주중이면 아침 7시에 알람을 울리고, 그렇지 않으면 아침 10시에 알람을 울려라.'로 조건을 바꿔도 되지 않을까요? 파이썬에서는 조건이 2개일 때 앞서 배운 `if`에 '그렇지 않으면'이라는 의미의 `else`를 추가해 다음 형식으로 표현합니다.

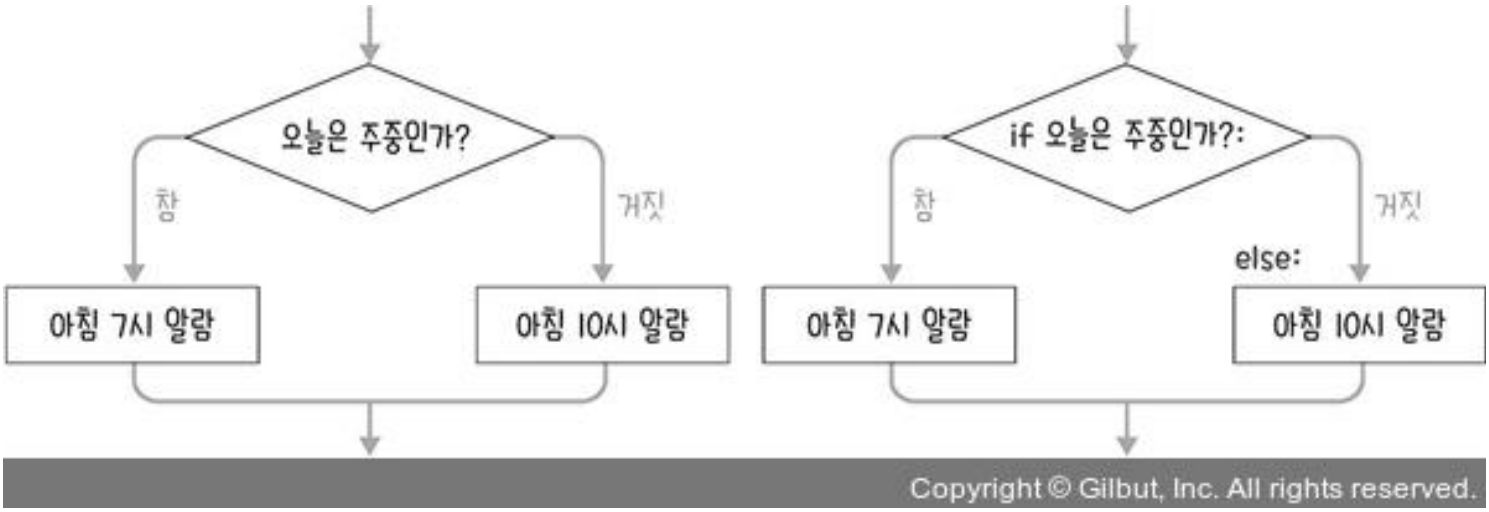
형식

```
if A: # 조건
    B # 조건을 충족할 때 실행할 내용
else:
    C # 조건을 충족하지 않을 때 실행할 내용
```

조건 A를 충족하면 B를 실행하고, 조건 A를 충족하지 않으면 C를 실행하는 구조입니다. 알람 설정 프로그램을 `if-else` 조건문 구조로 수정해 봅시다.

```
day = input('오늘은 주중입니까? 주말입니까? ')
if day == '주중':
    print('아침 7시입니다! 주인님 일어나세요!')
else:
    print('아침 10시입니다! 주인님 일어나세요!')
```

`else`는 '또 다른, 그 밖의'라는 뜻이 있습니다. 단어의 의미를 생각하며 형식을 살펴봅시다. 코드의 조건을 이해하기 쉽게 그림으로 나타냈습니다. 그림에서 마름모는 조건을 의미합니다. '오늘은 주중인가?'라는 조건에 대한 대답이 예(참)라면 왼쪽 화살표를 타고 내려와 아침 7시 알람을 실행합니다. 반면 대답이 아니요(거짓)라면 오른쪽 화살표를 따라 아침 10시 알람을 실행합니다.



▲ 그림 4-1 알람 설정 프로그램의 if-else 구조

이처럼 프로그램의 흐름(단계)을 순서대로 구조화해 나타낸 그림을 순서도라고 합니다. 순서도는 프로그램의 흐름을 한눈에 볼 수 있어서 프로그램을 작성하기 전에 프로그램 구조를 정리할 때 유용합니다.

> 제제쌤의 조언
> 순서도는 원래 사용하는 모양이 따로 있고, 포함해야 하는 조건도 있지만, 여기서는 간단하게 표현했습니다. 프로그램의 흐름을 도식화한 것이 순서도이고, 요소들을 화살표로 연결해 순서를 나타낸다고 알면 됩니다.

일주일에서 주중이 아닐 때는 주말밖에 없습니다. 이처럼 조건을 충족하지 않는 나머지 경우에 실행할 명령어가 있으면 `else`를 활용하는 것이 `if` 조건문을 여러 개 사용하는 것보다 효율적입니다. `else`는 '그렇지 않으면'이라는 의미를 담고 있어서 `if`처럼 조건을 붙일 필요가 없습니다. 단, `else` 옆에 콜론(:)이 붙는 점은 같으니 유의하세요! 콜론(`:`)을 생략하면 마찬가지로 `SyntaxError`가 발생합니다. 그리고 `else`에 속한 명령어도 반드시 들여쓰기를 해야 합니다.

```
day = input('오늘은 주중입니까? 주말입니까? ')
if day == '주중':
    print('아침 7시입니다! 주인님 일어나세요!')
else
    print('아침 10시입니다! 주인님 일어나세요!')
```

`실행결과`

```
else
    ^
SyntaxError: invalid syntax
```

4.2.2 if-else로 간단한 프로그램 만들기

`if-else`로 간단한 예제 프로그램을 작성해 보겠습니다.

차량 2부제

미세먼지가 심한 날은 미세먼지 저감을 위해 차량 2부제가 간헐적으로 시행됩니다. 짝수인 날에는 차량번호 끝자리가 짝수인 차량만 통행이 가능하고, 홀수인 날에는 차량번호 끝자리가 홀수인 차량만 통행이 가능합니다.

날짜를 숫자로 입력받아 짝수이면 '짝수번호 차량만 통행 가능합니다. 홀수번호 차주는 오늘 대중교통을 이용하세요.'를, 홀수이면 '홀수번호 차량만 통행 가능합니다. 짝수번호 차주는 오늘 대중교통을 이용하세요.'를 출력하게 프로그래밍해 보세요.

> 힌트 짝수와 홀수인 조건을 어떻게 만들지 고민된다면 짝수와 홀수의 정의를 생각해 보세요. 짝수는 2로 나누었을 때 나머지가 0인 수이고, 홀수는 2로 나누었을 때 나머지가 1인 수입니다. 나머지를 구하는 연산자는 2장에서 배웠죠? 잘 생각해 보고 프로그램을 작성한 후에 책과 비교해 보세요.

```
date = int(input('오늘은 며칠입니까? '))
if date % 2 == 0:
    print('짝수번호 차량만 통행 가능합니다.')
    print('홀수번호 차주는 오늘 대중교통을 이용하세요.')
else:
    print('홀수번호 차량만 통행 가능합니다.')
    print('짝수번호 차주는 오늘 대중교통을 이용하세요.')
```

날짜를 입력받으려면 `input()` 명령어와 입력받은 값을 저장할 변수가 필요합니다. 날짜를 뜻하는 `date`라는 이름으로 변수를 만듭니다. i`nput()`으로 입력받은 날짜를 `int()`로 감싸 정수형으로 바꾸고 이를 변수 `date`에 저장하는 것까지 이제 손쉽게 할 수 있을 거예요.

다음으로 조건을 봅시다. 조건은 날짜가 짝수인 날과 홀수인 날입니다. 조건이 2개고 서로 배타적이니 `if-else` 조건문을 사용하면 됩니다. 조건 하나를 `if`로, 나머지 조건을 `else`로 합니다. 여기서는 짝수를 기준으로 작성합니다. 그럼 조건은 입력받은 날짜가 짝수, 즉 '`date`가 짝수인가?'가 되겠죠? 이 조건은 어떻게 작성하면 좋을까요? 힌트에서 보듯이 날짜를 2로 나눈 나머지가 0이면 입력한 날짜는 짝수입니다. 나머지 연산자는 `%`이므로 조건은 `date % 2 == 0`으로 작성할 수 있습니다.



Copyright © Gilbut, Inc. All rights reserved.

`if` 문에서 `date % 2`를 실행해 나머지를 구한 후 0과 같은지를 비교합니다. 여기서 '예'가 나오면 짝수번호 차량만 통행 가능하다고 출력하고, 그렇지 않으면 `else`로 넘어가 홀수번호 차량만 통행 가능하다고 출력하게 됩니다.

반대로 홀수를 기준으로 프로그램을 작성해도 됩니다. 홀수를 기준으로 프로그램을 작성하면 `date % 2`가 같은지 비교하는 값을 1로 바꾸면 되겠죠? 결과는 동일합니다.

```
print('미세먼지 저감 조치에 따른 차량 2부제를 시행합니다!')
date = int(input('오늘은 며칠입니까? '))
if date % 2 == 1:
    print('홀수번호 차량만 통행 가능합니다')
    print('짝수번호 차주는 오늘 대중교통을 이용하세요.')
else:
    print('짝수번호 차량만 통행 가능합니다.')
    print('홀수번호 차주는 오늘 대중교통을 이용하세요.')
```

조금 더 친절하게 첫 줄에 미세먼지 저감을 위한 차량 2부제를 실행한다고 출력하게 했습니다. 다양한 값을 넣으며 테스트해 보세요.

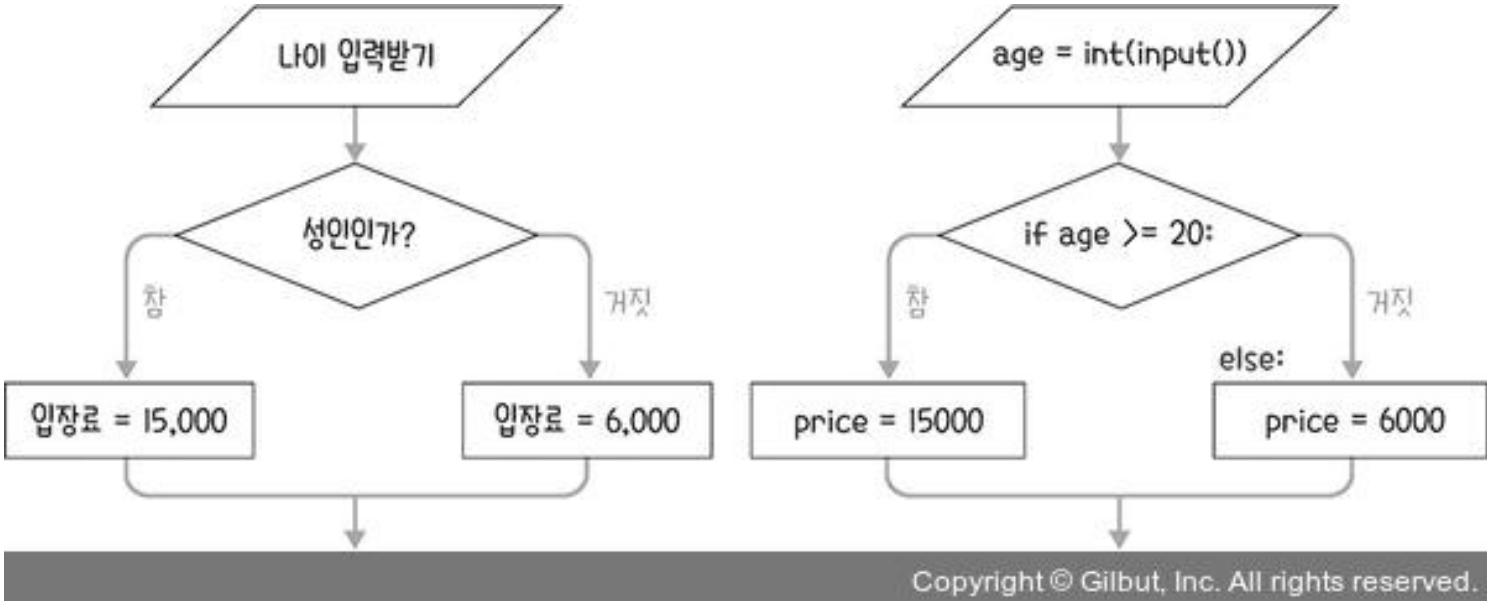
실행결과

```
20을 입력했을 때
미세먼지 저감 조치에 따른 차량 2부제를 시행합니다!
오늘은 며칠입니까? 20
짝수번호 차량만 통행 가능합니다.
홀수번호 차주는 오늘 대중교통을 이용하세요.

17을 입력했을 때
미세먼지 저감 조치에 따른 차량 2부제를 시행합니다!
오늘은 며칠입니까? 17
홀수번호 차량만 통행 가능합니다.
짝수번호 차주는 오늘 대중교통을 이용하세요.
```

무인계산기

프로그램을 하나 더 작성해 봅시다. 목욕탕에 무인계산기를 설치하려고 합니다. 나이를 입력받아 성인일 경우 15,000원, 미성년자일 경우 6,000원의 입장료를 내도록 안내하는 프로그램입니다. 왼쪽 그림처럼 순서도를 작성할 수 있겠죠. 그리고 나이를 저장하는 변수는 `age`, 입장료를 저장하는 변수는 `price`로 한다면 오른쪽 순서도처럼 코드로 표현할 수 있습니다.



▲ 그림 4-2 무인계산기 입장료 안내 순서도

나이(숫자)를 입력받지만, `input()`으로 입력받은 값은 문자열이므로 `int()`로 감싸 정수형으로 바꾼 후 변수 `age`에 저장합니다. 조건이 성인과 미성년자로 나뉘므로 `if-else` 형식으로 나이가 20살 이상이면 성인, 그렇지 않으면 미성년자를 표현합니다.

TIP 순서도에서 입출력은 평행사변형으로 표현합니다. 따라서 나이를 입력받는 부분을 평행사변형으로 표현했습니다.

순서도를 참고해 코드를 작성해 봅시다. 적절한 입력 안내문과 가격 안내문을 포함하겠습니다.

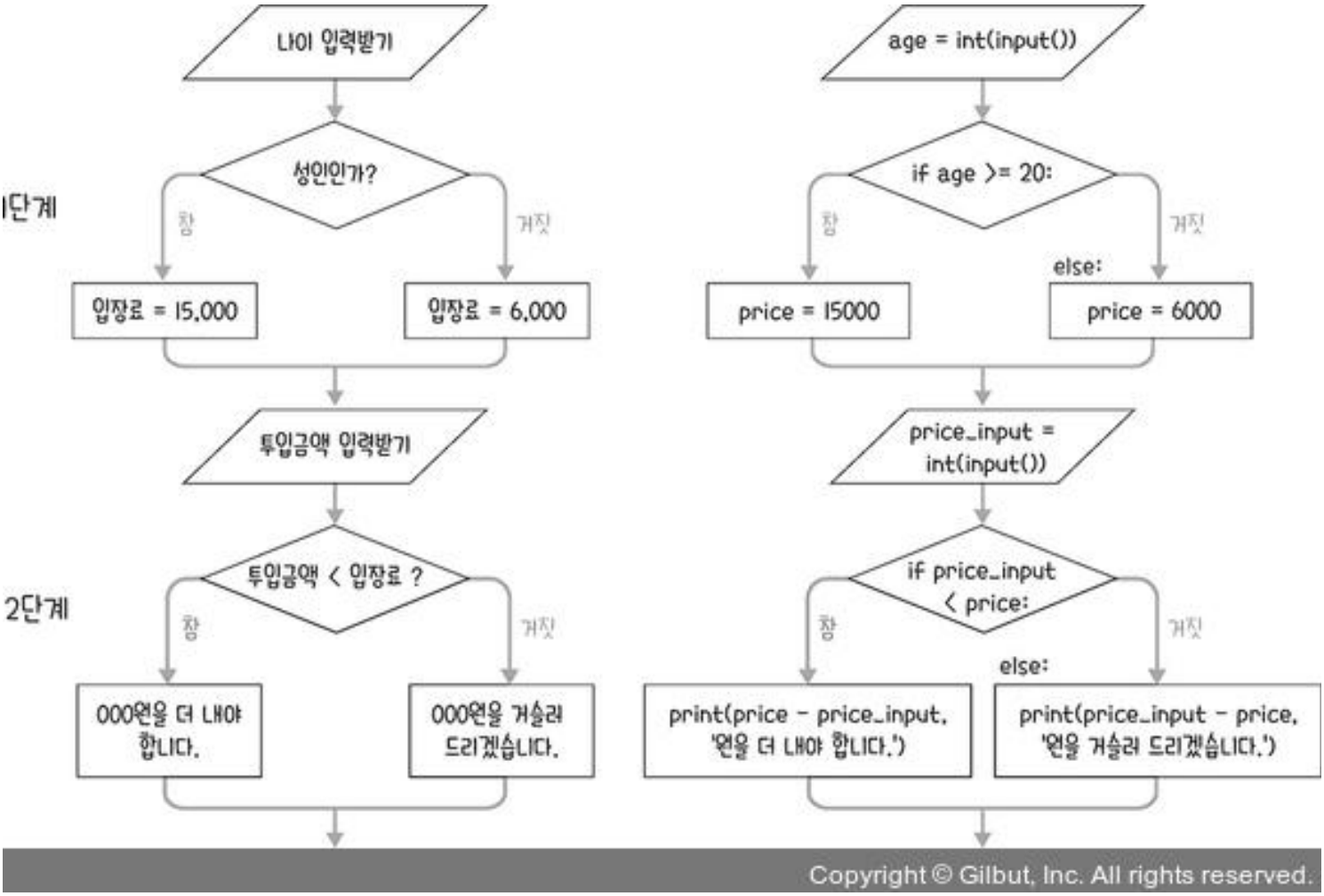
```
age = int(input('나이가 어떻게 되세요? '))
if age >= 20:
    print('성인은 15,000원입니다.')
    price = 15000
else:
    print('미성년자는 6,000원입니다.')
    price = 6000
```

코드를 여러 번 재실행해 미성년자 나이도 넣고, 성인 나이도 넣어 테스트해 보세요. 나이에 따라 다른 문장이 출력될 겁니다.

TIP 앞의 프로그램은 성인(`age >= 20`)을 기준으로 작성했지만, 미성년자(`age < 20`)를 기준으로 해도 조건 순서만 다를 뿐 결과는 같습니다.

이렇게 입장료를 알려 주고 나면 무인계산기의 투입구가 열리고 현금을 넣을 수 있다고 해 봅시다. 그러면 투입된 금액이 맞는지 확인해야죠. 투입금액이 입장료보다 적으면 부족한 금액을 출력하고, 입장료보다 많으면 거스름돈이 얼마인지 출력해 봅시다.

앞에 작성한 코드에 이어 투입금액 확인 부분을 추가해야겠네요. 순서도로 표현하면 다음과 같습니다. 프로그램 구조를 `if-else` 조건문 형식으로 표현하면 오른쪽 순서도와 같습니다. 투입금액을 저장하는 변수는 `price_input`이라고 합시다.



▲ 그림 4-3 무인계산기 프로그램의 구조

그림으로 보니 프로그램 구조가 잘 보이네요. 그림처럼 `if-else` 조건문은 총 2개가 필요합니다. 첫 번째 조건문에서는 나이를 조건으로, 두 번째 조건문에서는 입장료를 조건으로 하면 됩니다. 두 번째 조건문에서 입장료는 첫 번째 조건문의 나이에 따라 달라집니다. 나이에 따라 값이 바뀌니 역시 변수가 필요하죠. 앞에서 입장료를 저장한 변수 `price`를 사용하겠습니다.

고객이 투입금액을 입력하면 추가 금액을 요구하는 경우와 돈을 거슬러 주는 경우가 생깁니다. 투입금액 `price_input`이 입장료 `price`보다 적은 경우와 그렇지 않은 경우입니다. 먼저 투입금액이 입장료보다 적은 조건을 코드로 작성하면 `price_input < price`입니다. 그리고 차액은 더 큰 금액인 입장료에서 투입금액을 뺀 `price - price_input`이 되고요. 그렇지 않으면, 즉 투입금액이 입장료보다 많으면 `price_input`이 더 크므로 차액은 큰 숫자에서 작은 숫자를 뺀 `price_input - price`로 구합니다.

앞에서 작성한 1단계 코드에 이어 2단계 코드를 작성해 봅시다.

```
# 1단계 - 나이 입력받아 입장료 결정하기
age = int(input('나이가 어떻게 되세요? '))
if age >= 20:
    print('성인은 15,000원입니다.')
    price = 15000
else:
    print('미성년자는 6,000원입니다.')
    price = 6000

# 2단계 - 투입금액을 입력받아 입장료와 비교하기
price_input = int(input('입장료를 넣어 주세요. --> '))
if price_input < price:
    print(price-price_input, '원을 더 내야 합니다.')
else:
    print(price_input-price, '원을 거슬러 드리겠습니다.')
```

`price_input`에 투입금액을 저장하는 부분은 나이를 입력받는 부분과 유사하므로 참고해서 작성하면 됩니다.

결과는 ① 성인이 입장료보다 적은 금액을 투입한 경우 ② 성인이 입장료보다 많은 금액을 투입한 경우 ③ 미성년자가 입장료보다 적은 금액을 투입한 경우 ④ 미성년자가 입장료보다 많은 금액을 투입한 경우, 총 네 가지를 생각해 봅시다. 프로그램을 여러 번 실행하면서 다양한 숫자를 입력해 봅시다.

```
실행결과
나이 45, 투입금액 10000 입력 시
나이가 어떻게 되세요? 45
성인은 15,000원입니다.
입장료를 넣어 주세요. --> 10000
5000 원을 더 내야 합니다.

나이 6, 투입금액 25000 입력 시
나이가 어떻게 되세요? 6
미성년자는 6,000원입니다.
입장료를 넣어 주세요. --> 25000
19000 원을 거슬러 드리겠습니다.
```

그런데 입장료와 같은 금액을 넣으면 어떻게 될까요? 프로그램을 다시 실행해 나이 45, 투입금액 15000을 입력해 봅시다.

```
실행결과
나이가 어떻게 되세요? 45
성인은 15,000원입니다.
입장료를 넣어 주세요. --> 15000
0 원을 거슬러 드리겠습니다.
```

입장료와 같은 금액을 넣어도 돈을 거슬러 준다고 출력됩니다. 왜 이런 결과가 나왔는지 코드를 살펴봅시다. `if-else` 조건문에서 `if` 조건을 충족하지 않으면 모두 `else`가 됩니다. 따라서 이 코드에서 투입금액과 입장료가 같을 때, 즉 `price_input`와 `price`가 같은 값일 때는 `price_input < price` 조건을 충족하지 않으므로 `else`로 넘어가게 됩니다.

이 부분을 고쳐 봅시다. 조건을 바꿔 `price_input <= price` 또는 `price_input > price`로 수정하면 될까요? 이렇게 바꿔도 문제는 해결되지 않습니다. 결과가 궁금하다면 직접 코드를 수정하고 확인해 보세요. 바로 다음 절에서 이 문제를 해결해 보겠습니다.

1분 퀴즈

2 다음 코드를 해석한 것으로 옳지 <u>않은</u> 것을 고르세요.

```
a = int(input('숫자 입력: '))
if a < 0:
    a = -a
else:
    a *= 5
print(a)
```

① `a`는 정수를 저장하는 변수다.

② 음수가 입력되면 `a < 0` 조건을 충족하므로 결과는 양수로 바뀐다.

③ 양수가 입력되면 `a`에 5를 곱한 값이 출력된다.

④ 0이 입력되면 오류가 발생한다.

⑤ 마지막 줄의 `print(a)`는 항상 실행된다.

정답 및 해설

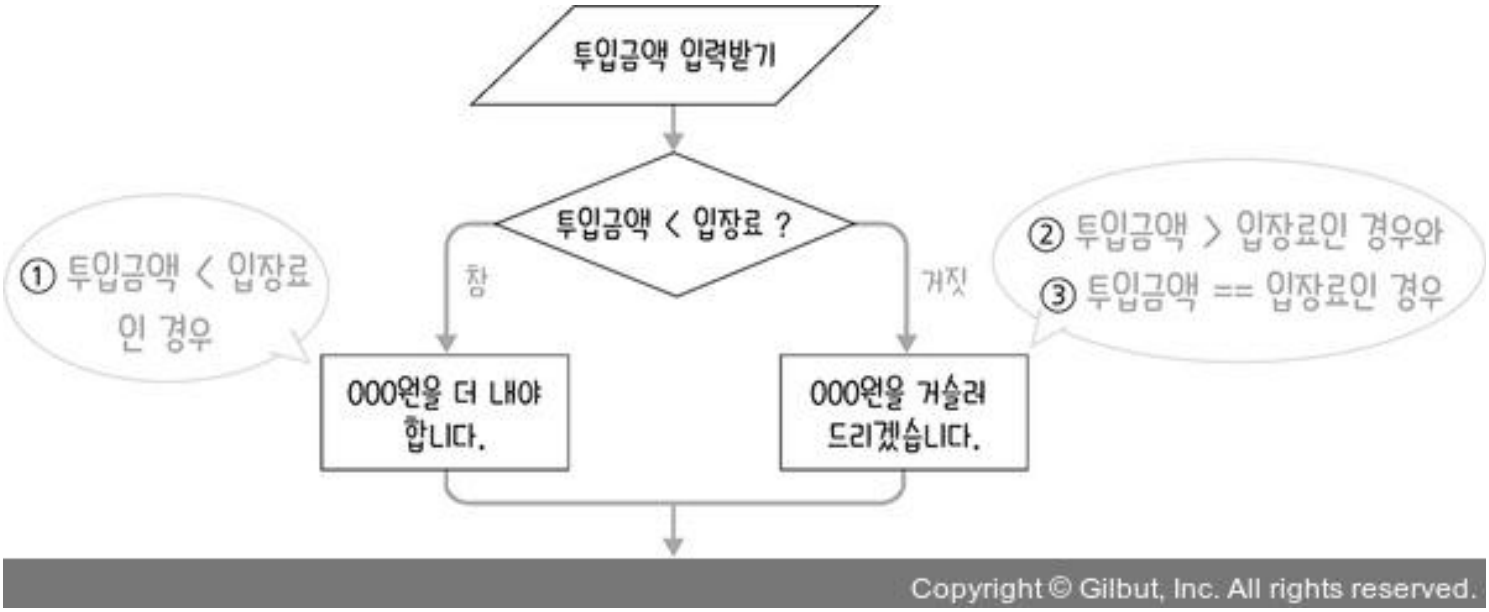
4.3

조건이 여러 개일 때: elif

4.3.1 조건이 3개일 때

무인계산기 프로그램에서 입장료에 맞게 정확한 금액을 넣었는데 왜 결과가 이상하게 나왔을까요? 표현할 수 있는 조건의 개수를 생각하면 원인을 알 수 있습니다. `if`는 조건을 충족하는 한 가지 경우만 표현할 수 있습니다. `if-else`는 조건을 충족하는 경우와 그렇지 않은 경우 두 가지를 표현할 수 있습니다.

문제 상황으로 돌아가 봅시다. 돈이 투입됐을 때 가능한 상황은 몇 가지인가요? ① 입장료 < 투입금액, ② 입장료 > 투입금액, ③ 입장료 == 투입금액, 총 세 가지입니다. 그래서 두 가지 조건을 표현하는 `if-else`로는 한계가 있을 수밖에 없습니다. 이를 그림으로 나타내면 다음과 같습니다.



▲ 그림 4-4 현재 무인계산기 프로그램의 구조

조건이 3개인 무인계산기에서는 `if`와 `else` 사이에 조건을 하나 더 추가하면 됩니다.

4.3.2 else if를 뜻하는 elif

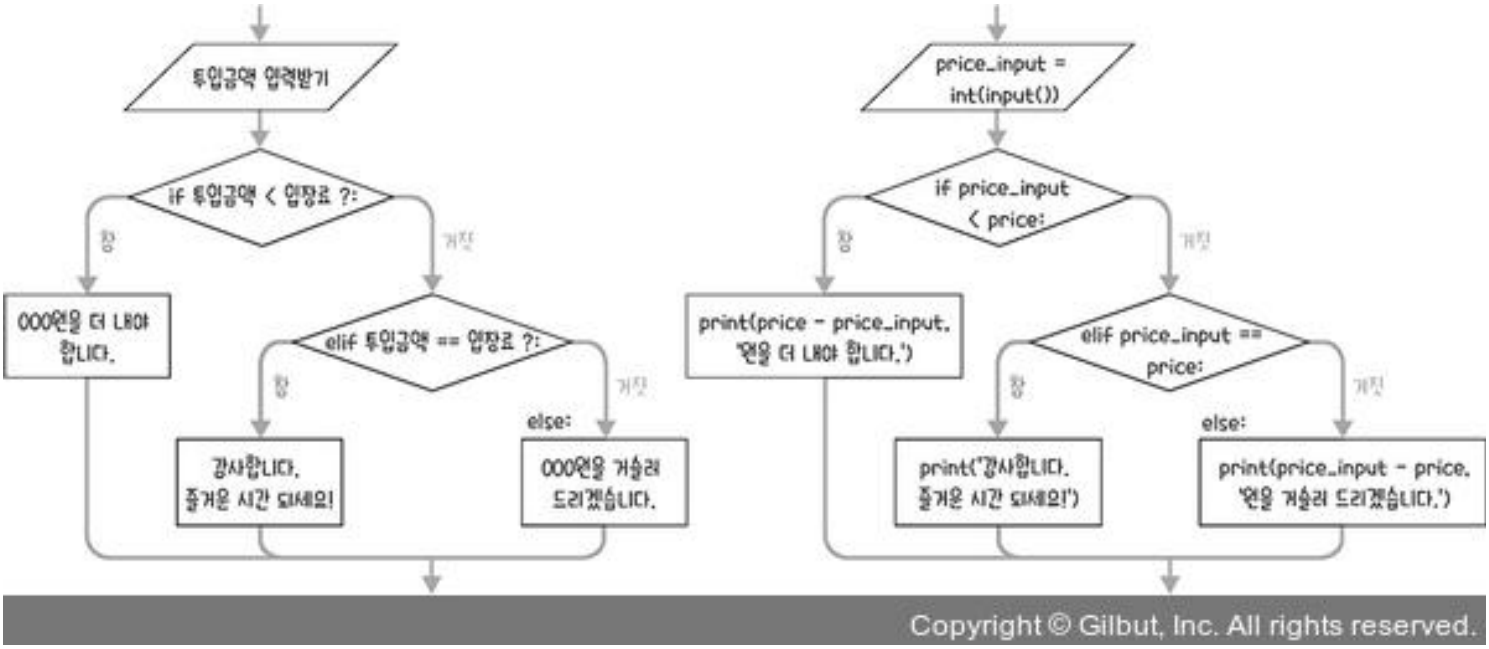
조건이 3개 이상, 즉 조건이 여러 개일 때는 `if-else` 구조에서 `if`와 `else` 사이에 '그렇지 않고 만약 ~라면 ~해라'라는 조건을 추가합니다. 파이썬에서는 이를 다음과 같이 표현합니다.

형식

```
if A:   # 조건
    B   # 조건 A를 충족할 때 실행할 내용
elif C: # 조건
    D   # 조건 A를 충족하지 않고 조건 C를 충족할 때 실행할 내용
else:
    E   # 조건 A, C를 모두 충족하지 않을 때 실행할 내용
```

`if`와 `else` 사이에 `elif`를 추가했습니다. `elif`는 `else`와 `if`가 합쳐진 단어입니다. `elif`가 추가된 조건문은 조건 A를 충족하지 않으면 B를 실행하지 않고 바로 다음 줄인 `elif`로 가서 조건 C를 충족하는지를 확인합니다. 여기서도 조건을 충족하지 않으면 `else`로 가서 E를 실행합니다.

무인계산기 프로그램으로 돌아가 봅시다. 투입금액(`price_input`)과 입장료(`price`)가 같은 조건을 `if`와 `else` 사이에 넣어 봅시다. `elif`는 `if` 조건이 충족되지 않을 때 실행되므로 `if`가 거짓일 때에 추가돼야 합니다. 그리고 `if`도 거짓이고, `elif`도 거짓일 때 비로소 `else`가 실행됩니다. 그러면 프로그램 구조는 다음과 같이 바뀝니다.



▲ 그림 4-5 elif를 추가한 무인계산기 프로그램 구조

추가한 조건을 코드에 반영해 봅시다.

```
# 1단계
age = int(input('나이가 어떻게 되세요? '))
if age >= 20:
    print('성인은 15,000원입니다.')
    price = 15000
else:
    print('미성년자는 6,000원입니다.')
    price = 6000

# 2단계
price_input = int(input('입장료를 넣어 주세요. --> '))
if price_input < price:
    print(price-price_input, '원을 더 내야 합니다.')
elif price_input == price:
    print('감사합니다. 즐거운 시간 되세요!')
else:
    print(price_input-price, '원을 거슬러 드리겠습니다.')
```

```
실행결과
나이가 어떻게 되세요? 45
성인은 15,000원입니다.
입장료를 넣어 주세요. --> 15000
감사합니다. 즐거운 시간 되세요!
```

투입금액이 입장료와 같으면 첫 번째 `if` 조건은 충족하지 않고 두 번째 `elif` 조건을 충족하므로 `print('감사합니다. 즐거운 시간 되세요!')`를 실행하고 종료합니다.

4.3.3 elif로 간단한 프로그램 만들기

`elif`로 간단한 예제 프로그램을 작성해 보겠습니다.

무료배송 수정하기

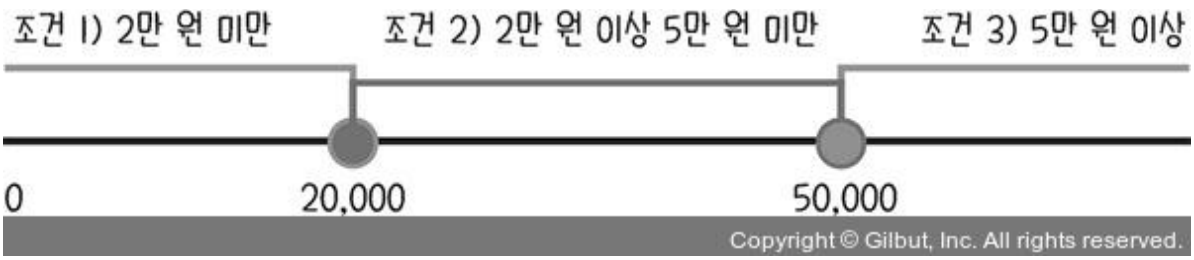
물건을 주문할 때 최저 구매금액이 조건으로 붙는 경우가 있습니다. 최저 구매금액이 2만 원이고, 2만 원 이상 5만 원 미만이면 구매금액에 배송비 2500원이 추가되며, 5만 원 이상이면 무료배송을 제공한다고 합시다. 구매금액을 입력받아 금액이 2만 원 미만이면 '주문금액이 부족합니다.'를 출력합니다. 2만 원 이상, 5만 원 미만이면 '배송비 2500원이 추가됩니다.'를, 5만 원 이상이면 '무료배송됩니다.'를 출력합니다. 그리고 모든 경우에 대해 실제 결제금액을 알려 주고 프로그램을 종료하겠습니다. 앞에서 작성한 무료배송 프로그램을 참고해 프로그램을 작성해 봅시다.

> 제제쌤의 조언
> 해답을 보기 전에 먼저 어떤 변수가 있어야 하는지, 조건은 몇 개인지, 각각의 조건은 어떻게 되는지를 생각해 본 후 작성하세요!

```
price = int(input('구매금액은? '))
if price < 20000:
    print('주문금액이 부족합니다.')
elif price < 50000:
    print('배송비 2500원이 추가됩니다.')
    price += 2500
else:
    print('무료배송됩니다.')
print('최종 결제금액은', price, '원입니다.')
```

```
실행결과
구매금액은? 20000
배송비 2500원이 추가됩니다.
최종 결제금액은 22500 원입니다.
```

먼저 구매금액을 입력받아 변수에 저장합니다. 그런 다음 조건에 따라 조건문을 만듭니다. 조건은 금액이 2만 원 미만일 때, 2만 원 이상 5만 원 미만일 때, 5만 원 이상일 때 세 가지이므로 `elif`를 활용해야 모든 조건을 표현할 수 있습니다. 조건 구간은 다음과 같습니다.



▲ 그림 4-6 조건 범위

첫 번째 조건은 '구매금액이 2만 원 미만'이므로 `price < 20000`이 되고, 실행할 명령어는 `print('주문금액이 부족합니다.')`입니다. 두 번째 조건은 2만 원 이상이면서 5만 원 미만에 해당하므로 `20000 <= price < 50000`일까요? 아닙니다. `elif`에는 이미 첫 번째 조건인 '2만 원 미만'을 충족하지 않는다는 의미가 포함되어 있으므로 두 번째 조건에서 '2만 원 이상(`20000 <= price`)'을 비교할 필요가 없습니다. 세 번째 조건인 5만 원 이상도 `price >= 50000`이라고 조건을 적지 않아도 됩니다. `else`에 이미 '첫 번째 조건과 두 번째 조건을 충족하지 않는다'는 의미가 들어 있기 때문이죠.

그리고 최종 결제금액은 항상 알려 주어야 하기 때문에 들여쓰기를 하지 않고 조건문 밖에서 작성해야 합니다. 따라서 조건 충족 여부와 상관없이 `print('최종 결제금액은', price, '원입니다.')`가 실행됩니다.

차량 2부제 수정하기
앞에서 작성한 차량 2부제 프로그램도 다시 살펴봅시다.

```
print('미세먼지 저감 조치에 따른 차량 2부제를 시행합니다!')

date = int(input('오늘은 며칠입니까? '))
if date % 2 == 1:
    print('홀수번호 차량만 통행 가능합니다.')
    print('짝수번호 차주는 오늘 대중교통을 이용하세요.')
else:
    print('짝수번호 차량만 통행 가능합니다.')
    print('홀수번호 차주는 오늘 대중교통을 이용하세요.')
```

날짜를 입력받아 홀짝을 판단한 후 차량 2부제를 안내했습니다. 이 프로그램에 45를 입력하면 어떻게 될까요? 직접 실행해 봅시다. 앞서 저장한 코드를 다시 불러와도 되고, 그리 길지 않은 코드이니 새로 작성하면서 예제를 복습해도 좋습니다.

```
실행결과
미세먼지 저감 조치에 따른 차량 2부제를 시행합니다!
오늘은 며칠입니까? 45
홀수번호 차량만 통행 가능합니다.
짝수번호 차주는 오늘 대중교통을 이용하세요.
```

45를 입력하면 당연히 홀수번호 차량이 통행 가능하다는 메시지가 출력되죠. 하지만 1년 중 45일이 있는 달은 없습니다. 날짜에서 가장 큰 숫자는 31입니다. 따라서 31을 초과한 숫자를 입력하면 입력이 잘못 됐음을 안내해야 합니다. 31을 초과한 숫자가 입력되면 '올바른 날짜를 입력하세요.'라고 출력해 봅시다.

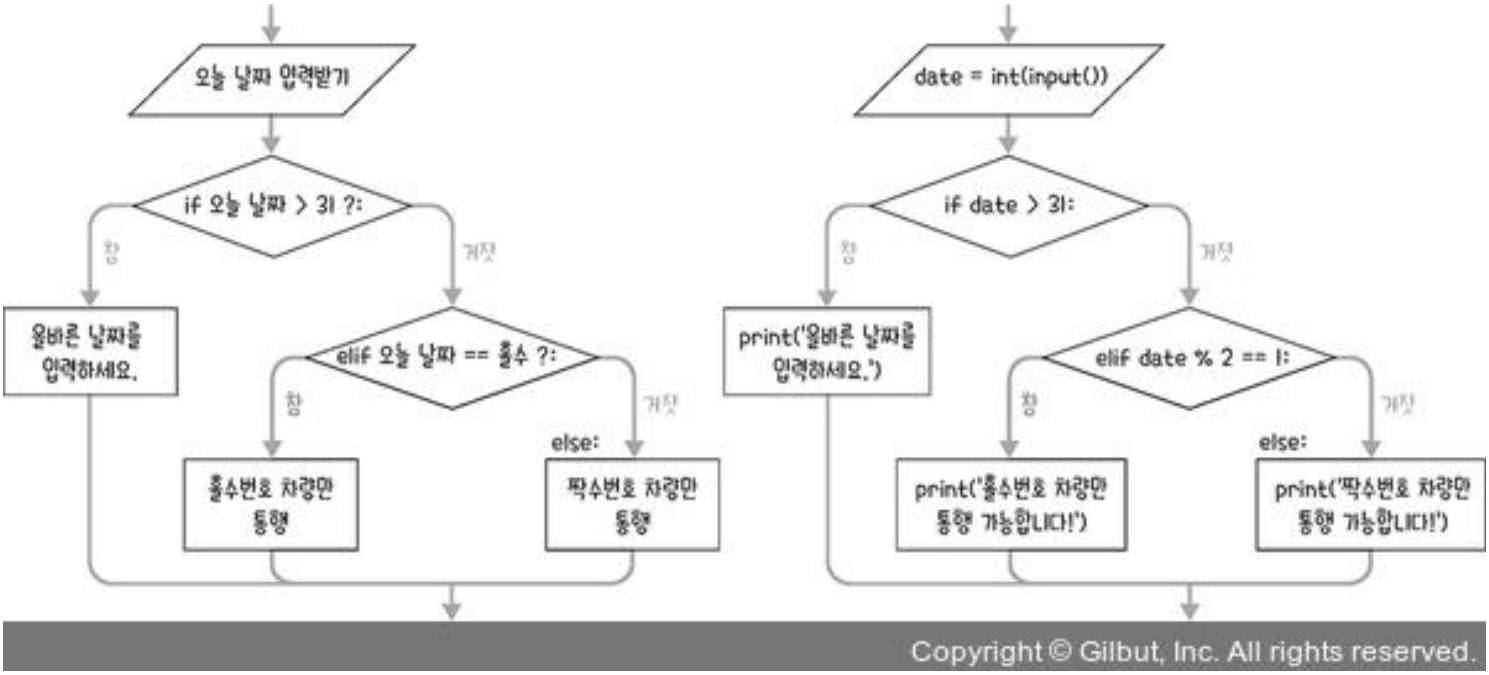
> 제제쌤의 조언
> 조건과 각 조건의 범위가 어떻게 되는지 생각해 보세요. 먼저 스스로 작성해 본 후 책의 코드를 확인하기 바랍니다.

```
print('미세먼지 저감 조치에 따른 차량 2부제를 시행합니다!')

date = int(input('오늘은 며칠입니까? '))
if date > 31:
    print('올바른 날짜를 입력하세요.')
elif date % 2 == 1:
    print('홀수번호 차량만 통행 가능합니다.')
    print('짝수번호 차주는 오늘 대중교통을 이용하세요.')
else:
    print('짝수번호 차량만 통행 가능합니다.')
    print('홀수번호 차주는 오늘 대중교통을 이용하세요.')
```

홀수와 짝수를 판단하기 전에 먼저 31을 초과한 숫자가 입력되면 올바른 날짜를 입력하라는 문구를 출력해야 합니다. 31을 초과하는지를 먼저 확인하는 이유는 입력값이 유효한지 검증하기 위해서입니다. 유효하지 않은 값이 입력되면 홀수와 짝수를 판단할 필요가 없습니다. 홀짝 판단을 먼저 한 후에 유효하지 않은 값임을 알게 됐을 때는 프로그램이 이미 상당 부분 진행된 상태입니다. 유효한 값만으로 홀짝을 판단하는 것이 순서에 맞고, 더 효율적입니다.

그래서 유효성을 검증하는 `date > 31`을 첫 번째 조건으로 사용합니다. 첫 번째 조건을 충족한다면 올바른 입력이 아니란 의미죠. 역으로 첫 번째 조건을 충족하지 않는다면 올바른 입력이란 뜻입니다. 따라서 두 번째, 세 번째 조건에서 홀수와 짝수를 판단해 알맞은 안내문을 출력합니다. 이를 순서도로 표현하면 다음과 같습니다.



▲ 그림 4-7 차량 2부제 프로그램의 순서도

앞에서 배웠듯이 `elif`는 '`date > 31`을 충족하지 않는다'는 의미를 포함하므로 '31 이하의 숫자 중 홀수'를 의미합니다. `else`도 `date > 31`과 `date % 2 == 1`에 모두 해당하지 않는 경우이므로 '31 이하의 숫자 중 짝수'를 의미합니다. `if`를 활용해 입력값의 유효성을 검증하는 방법은 실제 프로그래밍에서도 매우 흔하게 사용합니다. 효율적인 프로그램을 만드는 데 도움이 되니 구조를 잘 익혀 두기 바랍니다.

조건문은 다양한 방법으로 만들 수 있습니다. 반드시 `date > 31`이 첫 번째 조건으로 오지 않더라도 조건을 잘 설계하기만 하면 같은 결과가 나오는 프로그램을 작성할 수 있습니다. 다음 절에서 좀 더 자세히 알아보겠습니다.

NOTE 조건이 3개 이상일 때

조건이 3개가 아니라 4개 또는 그 이상일 수도 있습니다. 이런 경우 조건이 늘어날 때마다 `elif` 블록을 추가하면 됩니다.

```
if 조건 1:
    실행할 내용 1
elif 조건 2:
    실행할 내용 2
else:
    실행할 내용 3
```

```
if 조건 1:
    실행할 내용 1
elif 조건 2:
    실행할 내용 2
elif 조건 3:
    실행할 내용 3
else:
    실행할 내용 4
```

```
if 조건 1:
    실행할 내용 1
elif 조건 2:
    실행할 내용 2
elif 조건 3:
    실행할 내용 3
elif 조건 4:
    실행할 내용 4
…
else:
    실행할 내용 n
```

1분 퀴즈

3 목욕탕 입구에서 성별을 입력받아 남성이면 '<-- 남성전용탕은 왼쪽입니다.'를, 여성이면 '여성전용탕은 오른쪽입니다. -->'를 출력하려고 합니다. 그리고 남성이나 여성이 아닌 값을 입력받으면 '성별을 다시 입력해 주세요.'를 출력하게 합니다. 다음 프로그램에서 (가), (나)에 들어갈 코드를 올바르게 연결한 답을 각각 고르세요.

```
gender = input('성별을 입력해 주세요. ')
if gender == '남성':
    print('<-- 남성전용탕은 왼쪽입니다.')
(가)
    print('여성전용탕은 오른쪽입니다. -->')
(나)
    print('성별을 다시 입력해 주세요')
```

① (가) - `gender == '여성':`

② (가) - `else gender == '여성':`

③ (가) - `elif gender == '여성':`

④ (나) - `elif:`

⑤ (나) - `else:`

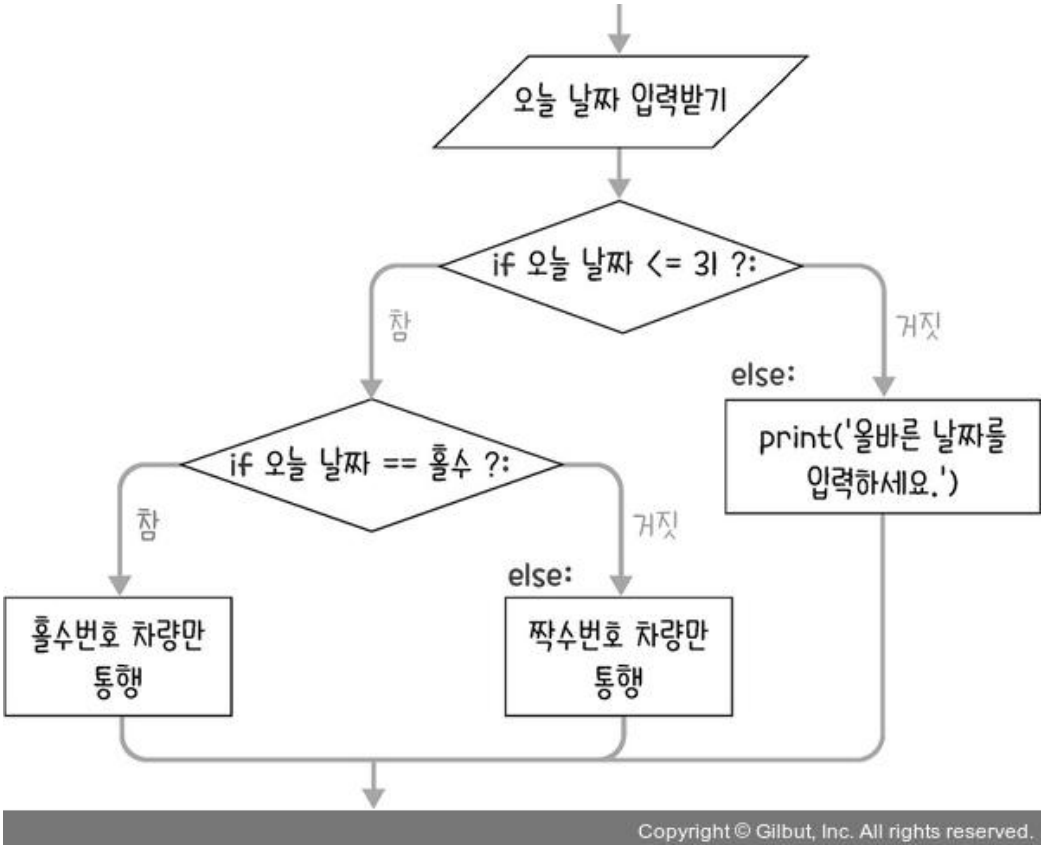
정답 및 해설

4.4

조건문 겹쳐 사용하기

4.4.1 조건문 안의 조건문

차량 2부제 프로그램은 `if` 문으로 먼저 유효하지 않은 값을 걸러낸 후에 홀수와 짝수를 판단했습니다. 순서를 바꿔 먼저 날짜가 31 이하일 때 홀수와 짝수를 판단하고, 31을 초과한 숫자를 걸러내면 어떨까요? 한번 시도해 봅시다! 순서도로 표현하면 다음과 같습니다.



▲ 그림 4-8 수정한 차량 2부제 프로그램의 순서도

`if-elif-else`와 결과는 동일하나 구조가 조금 다릅니다. `if` 문 안에 `if` 문이 있습니다. 이처럼 조건문 안에 조건문이 있는 형태를 중첩 조건문이라 합니다. 기존 코드의 구조만 바꾸면 되니 바로 작성해 봅시다.

```
print('미세먼지 저감 조치에 따른 차량 2부제를 시행합니다!')

date = int(input('오늘은 며칠입니까? '))
if date <= 31:
    if date % 2 == 1: → if 문 안에 if 문이 들어 있는 형태
        print('홀수번호 차량만 통행 가능합니다.')
        print('짝수번호 차주는 오늘 대중교통을 이용하세요.')
    else:
        print('짝수번호 차량만 통행 가능합니다.')
        print('홀수번호 차주는 오늘 대중교통을 이용하세요.')
else:
    print('올바른 날짜를 입력하세요.')
```

홀수와 짝수를 판단하는 `if-else` 문은 첫 번째 `if` 문(`if date <= 31`)이 충족될 때 실행되어야 하므로 들여쓰기를 합니다. 조건의 범위만 잘 파악하면 `if` 안에 `if`를 사용하는 중첩 조건문도 쉽게 작성할 수 있습니다.

4.4.2 여러 조건을 동시에 판단하는 and와 or

지금까지 살펴본 조건문에서는 한 번에 한 가지 조건만 판단했습니다. 그래서 두 가지 조건을 판단해야 할 때는 앞에서처럼 조건문 안에 조건문을 넣어 사용했습니다. 그런데 하나의 `if`로 여러 조건을 동시에 판단할 수도 있습니다.

여러 조건을 동시에 충족하는 and

백화점에서 사은품 행사를 하려고 합니다. 사은품 지급 대상은 구매금액이 10만 원 이상인 고객 중 마케팅 이용약관에 동의한 고객입니다. 조건이 구매금액과 이용약관 두 가지입니다. 두 가지 조건을 동시에 충족해야 하는데 이런 경우는 어떻게 표현할까요?

여러 조건을 동시에 판단할 때는 `and`를 사용합니다. and는 '그리고'라는 의미죠. 따라서 '조건 A를 충족한다 그리고 조건 B를 충족한다', 즉 '조건 A와 조건 B를 동시에 충족한다'는 뜻이 되며, '조건 A `and` 조건 B' 형식으로 사용합니다.

`and` 형식에 맞춰 '구매금액이 10만 원 이상인 고객 중 마케팅 이용약관에 동의했는가?'를 조건문으로 작성하면 다음과 같습니다.

```
if 구매금액 >= 100000 and 마케팅 이용약관 == 'Y':
```

앞의 조건문을 반영해 백화점 사은품 행사 프로그램을 코드로 작성해 봅시다. 백화점 사은품 행사는 알람 설정 프로그램과 구조가 비슷하므로 참고해서 작성하면 쉽습니다.

```
consumption = int(input('구매금액은? '))
marketing_yn = input('마케팅 이용약관에 동의했습니까? (Y/N) ')

if consumption >= 100000 and marketing_yn == 'Y':
    print('사은품 지급 대상입니다.')
else:
    print('사은품 지급 대상이 아닙니다.')
```

```
실행결과
구매금액이 120000원이고, 마케팅 이용약관 동의 Y일 때
구매금액은? 120000
마케팅 이용 약관에 동의했습니까? (Y/N) Y
사은품 지급 대상입니다.

구매금액이 120000원이고, 마케팅 이용약관 동의 N일 때
구매금액은? 120000
마케팅 이용약관에 동의했습니까? (Y/N) N
사은품 지급 대상이 아닙니다.
```

구매금액 조건과 마케팅 이용약관 조건을 `and`로 묶어 `if` 문에 넣고 두 조건을 모두 충족하면 사은품 지급 대상이라고 안내합니다. 그 외 경우는 `else`로 표현합니다.

두 조건을 조합하면 다음 네 가지 경우가 가능합니다. 코드를 재실행하며 두 조건을 동시 충족할 때만 사은품 지급 대상이 되는지 확인해 보세요.

> 제제쌤의 조언
> 백화점의 사은품 지급 행사는 중첩 조건문으로도 구현할 수 있습니다. 동일한 수학문제를 푸는 다양한 방법이 있는 것처럼 프로그래밍 역시 구현하는 방법이 다양합니다. 다양한 구현 방법을 알아 두면 복잡한 조건이 얽힌 상황도 쉽게 해결할 수 있으니 폭넓게 공부하는 것을 추천합니다.

▼ 표 4-2 조건 조합에 따른 사은품 지급 여부

구매금액 10만 원 이상	마케팅 이용약관 동의 Y	사은품 지급 대상
X	X	X
X	O	X
O	X	X
O	O	O

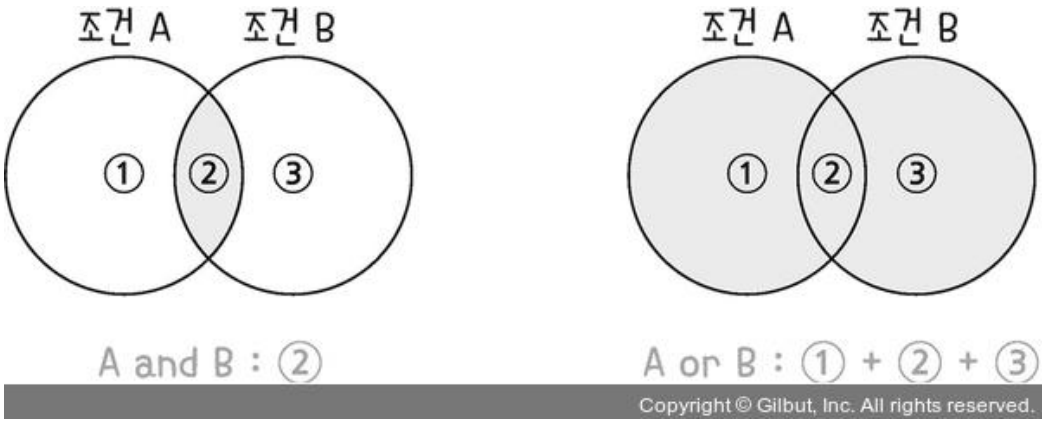
여러 조건 중 한 가지만 충족해도 되는 or

사은품 지급 대상은 두 조건을 모두 충족해야 하지만, 무료주차는 두 조건 중 하나만 충족해도 된다고 해 봅시다. 조합 결과는 다음과 같습니다.

▼ 표 4-3 조건 조합에 따른 무료주차 여부

구매금액 10만원 이상	마케팅 이용약관 동의 Y	무료주차 대상
X	X	X
X	O	O
O	X	O
O	O	O

두 조건을 모두 충족하지 않는 경우만 제외하고 모두 무료주차가 가능합니다. 이처럼 '조건 A 또는 조건 B'를 표현하는 방법은 `or`입니다. or는 '또는'이라는 뜻이죠. `or`도 `and`와 비슷하게 '조건 A `or` 조건 B' 형식으로 사용합니다. 다음 그림을 보면 조건 범위를 알 수 있습니다.



▲ 그림 4-9 and와 or의 조건 범위

그러면 `or`를 사용해 무료주차 여부를 판단하는 프로그램을 작성해 봅시다.

```
consumption = int(input('구매금액은? '))
marketing_yn = input('마케팅 이용약관에 동의했습니까? (Y/N) ')

if consumption >= 100000 or marketing_yn == 'Y':
    print('무료주차 대상입니다.')
else:
   print('무료주차 대상이 아닙니다.')
```

```
실행결과
구매금액은? 5000
마케팅 이용약관에 동의했습니까? (Y/N) Y
무료주차 대상입니다.
```

이번에는 구매금액 조건과 마케팅 이용약관 조건을 `or`로 묶어 `if` 문에 넣고, 둘 중 하나라도 충족하면 무료주차 대상이라고 안내합니다. 그 외 경우는 `else`로 넘깁니다.

이처럼 같은 결과의 프로그램이더라도 구조를 다양하게 구현할 수 있습니다. 정말 사람이 생각하는 대로 작성하는 거죠. 논리에 허점이 없고 정확하게만 표현한다면 모두 답이 될 수 있습니다!

TIP 동의 여부를 Y 또는 N으로 입력했는데 책과 다른 결과가 나와서 당황할 수 있습니다. 그렇다면 대문자 Y, N이 아니라 소문자 y, n을 입력했는지 확인해 보세요. 조건문의 조건식을 만들 때 대문자 'Y'만 조건에 넣었습니다. 만약 소문자 y도 조건에 포함하고 싶다면 `consumption >= 100000 or marketing_yn == 'Y' or marketing_yn == 'y'`처럼 `or`로 조건 하나를 더 연결하면 됩니다.

1분 퀴즈

4 다음 프로그램을 보고 옳지 않은 것을 고르세요.

```
a = int(input('숫자 입력: '))
if a > 50:
    if a % 3 == 0:
        print('A')
    elif a % 3 == 1:
        print('B')
    else:
        print('C')
else:
    if a % 2 == 0:
        print('D')
    else:
        print('E')
```

① 51을 입력하면 A가 출력된다.

② 92를 입력하면 B가 출력된다.

③ 86을 입력하면 C가 출력된다.

④ 44를 입력하면 D가 출력된다.

⑤ 15를 입력하면 E가 출력된다.

정답 및 해설

4

마무리

이 장에서 배운 내용을 정리해 보겠습니다.

1 제어구조 프로그램의 흐름이자 프로그램 안에서 명령어가 실행되는 순서

① 순차구조 위에서 아래의 순서로 실행되는 구조

② 선택구조 조건 충족 여부에 따라 실행할 명령어가 달라지는 구조

2 조건문 조건 충족 여부에 따라 실행되는 내용이 다른 선택구조를 파이썬 코드로 표현한 것

3 조건문의 형식

구분	형식
조건이 하나일 때	`if A:` `  B`

조건을 충족할 때와 충족하지 않을 때	`if A:`
	`  B`
	`else:`
	`  C`

조건이 여러 개일 때	`if A:`
	`  B`
	`elif C:`
	`  D`
	`  else:`
	`  E`

4 비교 연산자 두 값을 비교하는 연산자

비교 연산자	의미	비교 연산자	의미
`a > b`	a가 b보다 큰가?	`a <= b`	a가 b보다 작거나 같은가?
`a < b`	a가 b보다 작은가?	`a == b`	a와 b는 같은가?
`a >= b`	a가 b보다 크거나 같은가?	`a != b`	a와 b는 다른가(=같지 않은가)?

5 여러 개의 조건을 동시에 판단하는 연산자

연산자	의미
`A and B`	A도 충족하고 B도 충족한다면(A와 B를 동시에 충족한다면)
`A or B`	A를 충족하거나 B를 충족한다면(A 또는 B 둘 중 하나라도 충족한다면)

Self Check

1. 독감인지 확인하는 프로그램을 작성하세요. 기침이 있으면 Y로, 기침이 없으면 N으로 입력받고 체온도 입력받습니다(체온은 실수형으로 입력받는다고 가정합니다). 기침이 있고, 체온이 38.5도 이상이면 '독감일 수 있습니다.'를 출력하세요.
2. 분식집에 메뉴가 김밥류와 라면류 두 종류가 있습니다. 김밥류에는 야채김밥(2,500원), 참치김밥(3,500원)이 있고, 라면류에는 기본라면(3,500원), 떡라면(4,000원), 만두라면(4,000원)이 있습니다. 메뉴로 '김밥'을 입력받으면 김밥류의 하위 메뉴 2가지를 출력하고, '라면'을 입력받으면 라면류의 하위 메뉴 3가지를 출력하는 프로그램을 작성하세요. 입력받은 값이 '김밥'도 '라면'도 아니면 '올바른 메뉴를 입력해 주세요.'라고 출력하세요.
3. 0점부터 100점 사이의 점수를 입력받아 80점 이상이면 'A 등급', 40점 이상 80점 미만이면 'B 등급', 40점 미만이면 'C 등급'을 출력하는 프로그램을 작성하세요. 그리고 0 미만의 음수나 100 초과의 숫자가 입력되면 '잘못된 숫자를 입력했습니다.'를 출력하세요.

정답 및 해설